

Lab 20 - Control Plane Anatomy and etcd Backup/Restore

Field	Value
Lab id	Lab 20
Course	TGS-2023039343 Kubernetes and Cloud Native Associate (KCNA) Training v6.0
Day / Topic	Day 4 Cluster Architecture, Installation & Configuration
Duration	55 minutes
Namespace	kcna-lab20
Maps to	LO5 Demonstrate Kubernetes solution for a specific business problem A5 Demonstrate how the recommended IT solutions and components collectively address an existing business problem or need K5 Tools and techniques for solution architecture modelling

□ □ Safety contract - read this before Step 1

This lab looks at the machinery that keeps the classroom cluster alive. Three rules, and they are not negotiable:

1. Every control-plane operation in Sections 4.2 - 4.9 is read-only. `kubectl get`, `kubectl describe`, `kubectl get --raw` against health endpoints, and reading static Pod manifests. Nothing is created, patched, restarted or deleted outside `kcna-lab20`.
2. You will take a real etcd snapshot. You will NOT restore one. `etcdctl snapshot save` is a client read: it streams a point-in-time copy out of a *running* etcd without pausing it, locking it or modifying a byte. That is safe and you will do it. `snapshot restore` replaces the cluster's state and is not performed in this course.
3. The restore procedure is a reading exercise. It appears in `data/etcd-restore-runbook.md` and in Section 4.12 of this README, with the real commands, so you can learn them. Those blocks are fenced as text, not bash, and carry a DO NOT RUN banner. If you want to practise a restore, build your own throwaway single-node kind cluster on your own machine, with nobody else connected, and destroy it afterwards.

The same discipline applies to `kubeadm upgrade`. Section 4.13 teaches the *concepts and sequence*. Do not upgrade the class cluster.

1. Objective

By the end of this lab you will be able to:

1. Name every control-plane component, state what it is responsible for, and explain the one path by which all of them reach cluster state.
2. Explain static Pods: why the kubelet runs four control-plane components from a directory on disk, and how the resulting mirror Pods appear in the API server.
3. Read `/etc/kubernetes/manifests/etcd.yaml` field by field and extract, from the manifest alone, the four inputs `etcdctl` needs.

4. Take an etcd snapshot with `etcdctl snapshot save` and verify it with `snapshot status`, correctly choosing between the `etcdctl` and `etcdutl` binaries for your etcd version.
5. State what an etcd restore does not recover, and design a backup policy with explicit RPO, RTO, retention and off-node requirements.
6. Describe the restore and kubeadm upgrade procedures accurately without having executed either against a shared cluster.
7. Diagnose a control-plane fault from `kubecttl get --raw=/readyz?verbose` and the kubelet's view of a static Pod.

2. Prerequisites

- * A single-node kind cluster on Kubernetes v1.30+, provisioned by kubeadm (kind uses kubeadm internally, so its control plane is laid out exactly like a real kubeadm cluster).
- * A kubecttl context with cluster-admin rights - you will read kube-system and call cluster-scoped `--raw` endpoints.
- * Optional: docker access to the kind node container. Everything essential in this lab works without it; the two optional steps that use `docker exec` are marked and have kubecttl-only alternatives.
- * Labs 17-19 completed. Every object you created in those labs is sitting in the etcd keyspace you are about to snapshot.

```
cd courseware/labs/lab-20-cluster-architecture-etcd
pwd
```

```
.../courseware/labs/lab-20-cluster-architecture-etcd
```

Capture your node name once - nearly every command below uses it:

```
NODE=$(kubecttl get nodes -o jsonpath='{.items[0].metadata.name}')
echo "NODE=$NODE"
```

```
NODE=kcna-control-plane
```

3. Scenario

Kallang Freight has hired a second platform engineer for the TrackLane team, and the handover document has one glaring gap: nobody currently on the team can explain what actually runs the cluster, and the "backup strategy" is a wiki page that says *"AWS snapshots the volume"*.

The compliance work you did in Lab 18 made this urgent. If the cluster's state is lost, the `PersistentVolume` objects vanish even though the customs archive bytes survive - and nobody would know which claim owned which volume.

Your task this session: produce the control-plane section of the TrackLane technical blueprint. That means (a) a component inventory that names the real objects on this cluster, (b) a **verified** etcd snapshot proving the procedure works, and (c) a written backup policy with numbers in it. You will also read the disaster-recovery runbook - and you will not run it.

4. Step-by-step procedure

4.1 - Namespace and datasets

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab20 created
```

Look at the platform team's inventory:

```
cut -d, -f1,2,4 data/control-plane-inventory.csv | column -s, -t
```

component	deployment_kind	listen_ports
kube-apiserver	StaticPod	6443
etcd	StaticPod	2379;2380
kube-controller-manager	StaticPod	10257
kube-scheduler	StaticPod	10259
kubelet	SystemdUnit	10250;10248
kube-proxy	DaemonSet	10256
coredns	Deployment	53;9153
kindnet	DaemonSet	none
local-path-provisioner	Deployment	none

```
echo "rows=$(( $(wc -l < data/control-plane-inventory.csv) - 1 ))"
echo "static_pods=$(grep -c ',StaticPod,' data/control-plane-inventory.csv)"
```

```
rows=9
static_pods=4
```

Load both datasets into the cluster:

```
kubectl -n kcna-lab20 create configmap control-plane-inventory \
  --from-file=control-plane-inventory.csv=data/control-plane-inventory.csv
kubectl -n kcna-lab20 create configmap etcd-restore-runbook \
  --from-file=etcd-restore-runbook.md=data/etcd-restore-runbook.md
```

```
configmap/control-plane-inventory created
configmap/etcd-restore-runbook created
```

4.2 - See the control plane (read-only)

```
kubectl -n kube-system get pods -o wide
```

NAME NODE	NOMINATED NODE	READY READINESS GATES	STATUS	RESTARTS	AGE	IP
coredns-7db6d8ff4d-lc4nv		1/1	Running	0	96m	10.244.0.3
kcna-control-plane	<none>	<none>				
coredns-7db6d8ff4d-w8m2j		1/1	Running	0	96m	10.244.0.2
kcna-control-plane	<none>	<none>				
etcd-kcna-control-plane		1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>				
kindnet-8xq6r		1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>				
kube-apiserver-kcna-control-plane		1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>				
kube-controller-manager-kcna-control-plane		1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>				
kube-proxy-2wr7n		1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>				
kube-scheduler-kcna-control-plane		1/1	Running	0	96m	172.18.0.2
kcna-control-plane	<none>	<none>				

Two IP ranges are visible and the difference matters:

* 172.18.0.2 - the node's own IP. Every control-plane component uses hostNetwork: true, because the API server must be reachable before the Pod network exists. Bootstrapping order forces this.

* 10.244.0.x - the Pod network. CoreDNS is an ordinary Deployment and gets an ordinary Pod IP.

Now the mechanism that ties them together:

Component	Responsibility	Talks to etcd?	Leader-elected?
etcd	Consistent, replicated key-value store. All cluster state.	*is* the store	Raft (its own quorum)
kube-apiserver	REST front door. AuthN → AuthZ → Admission → validate → persist → watch.	Yes - the only component that does	No (stateless, scale horizontally)
kube-scheduler	Watches Pending Pods, filters and scores nodes, writes a binding.	No - through the apiserver	Yes
kube-controller-manager	~40 controller loops (Deployment, ReplicaSet, Node, PV/PVC binding, ServiceAccount, EndpointSlice...).	No - through the apiserver	Yes
cloud-controller-manager	Cloud-specific loops (LoadBalancer, node lifecycle, routes). Absent on kind.	No	Yes
kubelet	Node agent. Watches for Pods bound to its node, starts containers, reports status.	No	No
kube-proxy	Programmes node dataplane rules so Service VIPs reach Pods.	No	No

The single most important architectural fact on this page: *only the API server talks to etcd.* The scheduler does not. The controller-manager does not. Your kubectl does not. Every read and every write funnels through one process that enforces authentication, authorisation, admission control and validation. Break that invariant - for example by handing

an application a direct etcd client certificate - and every security control in Kubernetes is bypassed at once.

Confirm the health of each component from the API's own endpoint:

```
kubectl get --raw='/readyz?verbose'
```

```
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
[+]poststarthook/start-apiserver-admission-initializer ok
[+]poststarthook/generic-apiserver-start-informers ok
[+]poststarthook/rbac/bootstrap-roles ok
[+]poststarthook/scheduling/bootstrap-system-priority-classes ok
[+]poststarthook/start-cluster-authentication-info-controller ok
[+]poststarthook/start-kube-apiserver-identity-lease-controller ok
[+]shutdown ok
readyz check passed
```

The exact list of checks varies by version. The two to look for are [+]etcd ok and the final readyz check passed.

4.3 - Static Pods: where four of those Pods actually come from

Pick any control-plane Pod and ask who owns it:

```
kubectl -n kube-system get pod "etcd-$NODE" \
-o jsonpath='{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name}{"\n"}'
```

```
Node/kcna-control-plane
```

Owned by a Node, not by a Deployment, ReplicaSet, DaemonSet or StatefulSet. That is the signature of a mirror Pod. Confirm it:

```
kubectl -n kube-system get pod "etcd-$NODE" \
-o jsonpath='{.metadata.annotations}{"\n"}' | tr ',' '\n' | grep config
```

```
"kubernetes.io/config.hash":"9a4c1f7e0b3d2856ac9f4b17e05d3c82"
"kubernetes.io/config.mirror":"9a4c1f7e0b3d2856ac9f4b17e05d3c82"
"kubernetes.io/config.seen":"2026-09-05T03:11:02.417392514Z"
"kubernetes.io/config.source":"file"
```

kubernetes.io/config.source: file is the decisive field. This Pod did not come from the API server. The kubelet read it from a directory on disk and then created a read-only *mirror* of it in the API so that kubectl can see it.

Why the bootstrap works this way. The API server is itself a Pod. Something has to start it before the API server exists to be asked. That something is the kubelet, reading YAML from --pod-manifest-path (/etc/kubernetes/manifests) with no API server involved at all. This is also why:

- * You cannot kubectl delete a static Pod and have it stay deleted - the kubelet recreates the mirror within seconds. To stop a static Pod you move its file.

- * kubectl edit on a mirror Pod is pointless - the kubelet overwrites it from the file.

- * The four control-plane components restart automatically after a node reboot with no controller involved.

List all mirror Pods on this node:

```
kubectl -n kube-system get pods \
  -o jsonpath='{range .items[?(@.metadata.annotations.kubernetes\.io/config\.source=="file")]]{
    .metadata.name}{ "\n" }{end}'
```

```
etcd-kcna-control-plane
kube-apiserver-kcna-control-plane
kube-controller-manager-kcna-control-plane
kube-scheduler-kcna-control-plane
```

Four - matching static_pods=4 from the platform team's inventory in Step 4.1.

4.4 - Read the manifest directory (optional: needs Docker)

On kind the node is a Docker container, so the kubelet's manifest directory is inside it:

```
docker exec "$NODE" ls -l /etc/kubernetes/manifests
```

```
total 16
-rw----- 1 root root 2406 Sep  5 03:10 etcd.yaml
-rw----- 1 root root 3896 Sep  5 03:10 kube-apiserver.yaml
-rw----- 1 root root 3428 Sep  5 03:10 kube-controller-manager.yaml
-rw----- 1 root root 1463 Sep  5 03:10 kube-scheduler.yaml
```

Four files, four mirror Pods. Confirm the kubelet is watching that exact path:

```
docker exec "$NODE" grep -i staticPodPath /var/lib/kubelet/config.yaml
```

```
staticPodPath: /etc/kubernetes/manifests
```

No Docker? Skip this step. The mirror Pod in the API server carries the same spec, and `kubectl -n kube-system get pod etcd-$NODE -o yaml` shows you everything you need for Step 4.5.

4.5 - Read the etcd manifest field by field

Use whichever source you have. Docker:

```
docker exec "$NODE" cat /etc/kubernetes/manifests/etcd.yaml | sed -n '1,40p'
```

...or, equivalently and with no Docker:

```
kubectl -n kube-system get pod "etcd-$NODE" -o yaml \
  | grep -E 'image:|--(advertise-client-urls|cert-file|key-file|trusted-ca-file|data-
  dir|listen-client-urls|listen-peer-urls|initial-cluster)='
```

```
- --advertise-client-urls=https://172.18.0.2:2379
- --cert-file=/etc/kubernetes/pki/etcd/server.crt
- --data-dir=/var/lib/etcd
- --initial-cluster=kcna-control-plane=https://172.18.0.2:2380
- --key-file=/etc/kubernetes/pki/etcd/server.key
- --listen-client-urls=https://127.0.0.1:2379,https://172.18.0.2:2379
- --listen-peer-urls=https://172.18.0.2:2380
- --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
image: registry.k8s.io/etcd:3.5.15-0
```

Read it as an operator would:

Flag	Value	What it tells you
--listen-client-urls	https://127.0.0.1:2379, https://172.18.0.2:2379	Port 2379 serves clients. 127.0.0.1 is why an in-container etcdctl can use the loopback endpoint.
--listen-peer-urls	https://172.18.0.2:2380	Port 2380 is member-to-member Raft traffic. Never expose it.
--data-dir	/var/lib/etcd	The keyspace on disk. The actual WAL and snapshots live in /var/lib/etcd/member/.
--trusted-ca-file	/etc/kubernetes/pki/etcd/ca.crt	The CA etcdctl must present as --cacert.
--cert-file / --key-file	server.crt / server.key	The client credentials etcdctl uses as --cert / --key.
--initial-cluster	one member	A single-member Raft group. Production wants 3 or 5 - an odd number, because quorum is $\lceil n/2 \rceil + 1$.
image	registry.k8s.io/etcd:3.5.15-0	Note this version. It decides which binary you use in Step 4.8.

Record the image, you will need it:

```
ETCD_IMAGE=$(kubectl -n kube-system get pod "etcd-$NODE" \
-o jsonpath='{.spec.containers[0].image}')
echo "ETCD_IMAGE=$ETCD_IMAGE"
```

```
ETCD_IMAGE=registry.k8s.io/etcd:3.5.15-0
```

Why etcd's own TLS matters. etcd has no concept of Kubernetes RBAC. Anyone holding server.crt/server.key can read and write the entire cluster keyspace - every Secret in every namespace, in plaintext unless you have configured encryption at rest. Those files are the crown jewels. This is also why etcd runs on the control-plane node with hostNetwork and its listener bound where only the API server can reach it.

4.6 - Watch the scheduler and controller-manager coordinate

Both are leader-elected, and they hold their leadership in a Lease object you can read:

```
kubectl -n kube-system get leases
```

NAME	AGE	HOLDER
apiserver-lz3rd7qmtqvfbhvxwl4gk6nz2a	99m	apiserver-lz3rd7qmtqvfbhvxwl4gk6nz2a_0f1d6c9b-7a34-4e02-b5c8-91d47ea36f0c
kube-controller-manager	99m	kcna-control-plane_3b8e5d21-6c47-4a09-9e13-5f0c82ab7d64
kube-scheduler	99m	kcna-control-plane_c7d0a493-8f52-41be-b6a7-2e9d5310cf88

```
kubectl -n kube-system get lease kube-scheduler \
-o jsonpath='{.spec.holderIdentity}{ " renewed: "}{.spec.renewTime}{ "\n"}'
```

```
kcna-control-plane_c7d0a493-8f52-41be-b6a7-2e9d5310cf88 renewed: 2026-09-05T06:52:41.883412Z
```

The mechanism. In a 3-node control plane all three schedulers are running, but only the one holding this Lease acts. It renews the Lease every few seconds; if it dies, the Lease expires and another instance takes it. The Lease is an ordinary API object - so leader election is itself stored in etcd and mediated by the API server. This is why the KCNA blueprint keeps returning to the same sentence: everything is an object, and the API server is the only door.

Trace one full interaction end to end using the Pods you created in Lab 19 (or any Pod):

```
kubectl -n kcna-lab20 run trace-probe --image=busybox:1.36 --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "trace-probe", "image": "busybox:1.36", "command": [
    "sh", "-c", "echo scheduled; sleep 60" ], "resources": { "requests": { "cpu": "50m", "memory": "64Mi" },
    "limits": { "cpu": "200m", "memory": "128Mi" } }, "securityContext": { "allowPrivilegeEscalation": false,
    "privileged": false, "capabilities": { "drop": [ "ALL" ] } } } ] } }
```

```
pod/trace-probe created
```

```
sleep 5
kubectl -n kcna-lab20 get events --field-selector involvedObject.name=trace-probe \
  -o custom-columns='TIME:.lastTimestamp,SOURCE:.source.component,REASON:.reason,MESSAGE:.
message'
```

TIME	SOURCE	REASON	MESSAGE
2026-09-05T06:55:12Z	default-scheduler	Scheduled	Successfully assigned kcna-lab20/trace-probe to kcna-control-plane
2026-09-05T06:55:13Z	kubelet	Pulled	Container image "busybox:1.36" already present on machine
2026-09-05T06:55:13Z	kubelet	Created	Created container: trace-probe
2026-09-05T06:55:13Z	kubelet	Started	Started container trace-probe

Read the SOURCE column: default-scheduler decided *where*; kubelet did *what*. Two different components, communicating only through objects in the API server. Neither ever contacted the other.

```
kubectl -n kcna-lab20 delete pod trace-probe --ignore-not-found
```

```
pod "trace-probe" deleted
```

4.7 - Generate the concrete inventory

```
kubectl apply -f manifests/01-rbac-namespace-reader.yaml
kubectl apply -f manifests/02-inventory-job.yaml
```

```
serviceaccount/cp-inventory created
role.rbac.authorization.k8s.io/cp-inventory-reader created
rolebinding.rbac.authorization.k8s.io/cp-inventory-reader created
job.batch/cp-inventory created
```

```
kubectl -n kcna-lab20 wait --for=condition=Complete job/cp-inventory --timeout=120s
kubectl -n kcna-lab20 logs job/cp-inventory
```

```

job.batch/cp-inventory condition met
TrackLane control-plane inventory for node kcna-control-plane
generated from data/control-plane-inventory.csv
=====
COMPONENT          DEPLOYMENT  OBJECT OR PATH TO INSPECT
kube-apiserver      StaticPod   kube-system/pod/kube-apiserver-kcna-control-plane
etcd                StaticPod   kube-system/pod/etcd-kcna-control-plane
kube-controller-manager
plane              StaticPod   kube-system/pod/kube-controller-manager-kcna-control-
plane
kube-scheduler      StaticPod   kube-system/pod/kube-scheduler-kcna-control-plane
kubelet             SystemdUnit /etc/systemd/system/kubelet.service.d/10-kubeadm.conf
kube-proxy          DaemonSet   api-object:kube-system/daemonset/kube-proxy
coredns             Deployment  api-object:kube-system/deployment/coredns
kindnet             DaemonSet   api-object:kube-system/daemonset/kindnet
local-path-provisioner
provisioner         Deployment  api-object:local-path-storage/deployment/local-path-
provisioner
=====
inventory_rows=9
static_pod_components=4
namespace=kcna-lab20
NOTE: this Job has namespaced RBAC only and cannot read kube-system.

```

The generic dataset has become the exact object names for this cluster. Verify the least-privilege claim in that last line:

```

kubectl auth can-i list pods --namespace=kube-system \
  --as=system:serviceaccount:kcna-lab20:cp-inventory

```

no

```

kubectl auth can-i list pods --namespace=kcna-lab20 \
  --as=system:serviceaccount:kcna-lab20:cp-inventory

```

yes

Your kubectl can read the control plane; the workload you deployed cannot. That asymmetry is the correct default for platform tooling.

4.8 - Take an etcd snapshot

Choosing the method - and why.

Method	Verdict
kubectl -n kube-system exec into the etcd Pod	This is what we use. etcdctl ships *inside* the etcd container image and is not installed on the kind node or on your laptop. The TLS client certificates it needs are already mounted into that container. The operation goes through the API server, so it is subject to RBAC and lands in the audit log. It works identically on kind, on kubeadm bare metal, and on any cluster where you can exec into kube-system.
docker exec into the kind node	Kind-specific, bypasses the Kubernetes audit trail and RBAC entirely, requires Docker on your host - and etcdctl is not on the node anyway, so it would not work without extra installation. We use docker only in Step 4.9, to copy the finished file off the node, because /var/lib/etcd is a hostPath.

Method	Verdict
Install etcdctl on your laptop and dial the cluster	Requires exporting etcd's server certificate and key off the node. That is the one credential you should never move. Do not do this.

Find the etcd Pod and confirm etcdctl is present:

```
ETCD_POD=$(kubectl -n kube-system get pods -l component=etcd \
-o jsonpath='{.items[0].metadata.name}')
echo "ETCD_POD=$ETCD_POD"
kubectl -n kube-system exec "$ETCD_POD" -- etcdctl version
```

```
ETCD_POD=etcd-kcna-control-plane
etcdctl version: 3.5.15
API version: 3.5
```

Now take the snapshot. Read the four TLS arguments against the manifest flags from Step 4.5 - they are the same values:

```
kubectl -n kube-system exec "$ETCD_POD" -- etcdctl \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot save /var/lib/etcd/kcna-lab20-snapshot.db
```

```
{"level":"info","ts":"2026-09-05T07:02:18.441Z","caller":"snapshot/v3_snapshot.go:65","msg":
"created temporary db file","path":"/var/lib/etcd/kcna-lab20-snapshot.db.part"}
{"level":"info","ts":"2026-09-05T07:02:18.449Z","logger":"client","caller":"v3/maintenance.go:
212","msg":"opened snapshot stream; downloading"}
{"level":"info","ts":"2026-09-05T07:02:18.449Z","caller":"snapshot/v3_snapshot.go:73","msg":
"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info","ts":"2026-09-05T07:02:18.612Z","logger":"client","caller":"v3/maintenance.go:
220","msg":"completed snapshot read; closing"}
{"level":"info","ts":"2026-09-05T07:02:18.641Z","caller":"snapshot/v3_snapshot.go:88","msg":
"fetching snapshot","endpoint":"https://127.0.0.1:2379","size":"7.4 MB","took":"now"}
{"level":"info","ts":"2026-09-05T07:02:18.641Z","caller":"snapshot/v3_snapshot.go:97","msg":
"saved","path":"/var/lib/etcd/kcna-lab20-snapshot.db"}
Snapshot saved at /var/lib/etcd/kcna-lab20-snapshot.db
```

What just happened, and why it is safe. snapshot save is a *client* call to the running server's Maintenance API. etcd streamed a consistent copy of the key space at the current revision. It did not pause, lock, compact or modify anything. The cluster kept serving throughout - you can prove that:

```
kubectl get --raw='/readyz' && echo
```

```
ok
```

About the destination path. etcd's actual data lives in `/var/lib/etcd/member/` (wal/ and snap/). Writing a file to the *parent* directory `/var/lib/etcd/` is inert - etcd never reads it. We use that path because it is the one directory guaranteed to be both writable inside the container and retrievable from the node, since the manifest mounts it as a `hostPath`. In production you would mount a separate `/var/lib/etcd-backup` `hostPath` so that backups and live data never share a directory, and you would ship the file off the node immediately. Step 4.9 does exactly that, and Step 4.11 removes the file from the data directory.

4.9 - Verify the snapshot

A snapshot you have never verified is not a backup. snapshot status reads the file offline - it never contacts a server.

Which binary? This depends on your etcd version and is a common exam and interview point:

etcd version	snapshot save	snapshot status / snapshot restore
≤ 3.4	etcdctl	etcdctl
3.5.x	etcdctl	etcdctl - still works in etcdctl but prints a deprecation warning
≥ 3.6	etcdctl	etcdctl only - removed from etcdctl

The split exists because save talks to a **server** (a client operation) while status and restore operate on a **file** (an offline utility operation).

Try etcdctl first:

```
kubectl -n kube-system exec "$ETCD_POD" -- \
  etcdctl snapshot status /var/lib/etcd/kcna-lab20-snapshot.db -w table
```

```
+-----+-----+-----+-----+
|  HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+-----+-----+-----+-----+
| a41f7b2e |    38412 |        1187 |    7.4 MB |
+-----+-----+-----+-----+
```

If your etcd image is older than 3.5.x and etcdctl is not present, you will see:

```
OCI runtime exec failed: exec failed: unable to start container process: exec: "etcdctl":
executable file not found in $PATH: unknown
command terminated with exit code 126
```

In that case use the deprecated etcdctl form, which produces the same table:

```
kubectl -n kube-system exec "$ETCD_POD" -- \
  etcdctl snapshot status /var/lib/etcd/kcna-lab20-snapshot.db -w table
```

Deprecated: Use `etcdctl snapshot status` instead.

```
+-----+-----+-----+-----+
|  HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+-----+-----+-----+-----+
| a41f7b2e |    38412 |        1187 |    7.4 MB |
+-----+-----+-----+-----+
```

Read the four columns as an acceptance test:

Column	Meaning	What "bad" looks like
HASH	CRC of the file	Missing / command exits non-zero → corrupt file
REVISION	etcd's global revision at snapshot time	0 → the save failed
TOTAL KEYS	Number of keys captured	Single digits on a real cluster → wrong endpoint or empty database

Column	Meaning	What "bad" looks like
TOTAL SIZE	Bytes captured	A few KB → almost certainly a failed or truncated save

TOTAL KEYS on your cluster reflects everything you have created across all 20 labs so far. Confirm the file is real from the node's point of view, and copy it off (kind-specific, needs Docker):

```
docker exec "$NODE" ls -lh /var/lib/etcd/kcna-lab20-snapshot.db
```

```
-rw----- 1 root root 7.1M Sep  5 07:02 /var/lib/etcd/kcna-lab20-snapshot.db
```

```
docker cp "$NODE:/var/lib/etcd/kcna-lab20-snapshot.db" /tmp/kcna-lab20-snapshot.db
ls -lh /tmp/kcna-lab20-snapshot.db
```

```
Successfully copied 7.42MB to /tmp/kcna-lab20-snapshot.db
-rw----- 1 you  staff  7.1M  5 Sep 07:03 /tmp/kcna-lab20-snapshot.db
```

This copy step is the whole point of a backup. A snapshot sitting on the node whose disk you are protecting against is not a backup - it is a convenience. The policy in Step 4.10 makes off-node storage mandatory for exactly this reason.

4.10 - Write the backup policy down

```
kubectl apply -f manifests/03-backup-policy.yaml
```

```
configmap/etcd-backup-policy created
```

```
kubectl -n kcna-lab20 get configmap etcd-backup-policy \
-o jsonpath='{.data.policy\.yaml}' | sed -n '1,20p'
```

```
# Kallang Freight - TrackLane etcd backup policy
# Owner: Platform Engineering. Review: quarterly.
scope:
  cluster: tracklane-prod-sg1
  control_plane: kubeadm, 3 stacked etcd members
objectives:
  rpo_minutes: 30          # max acceptable data loss
  rto_minutes: 60          # decision -> /readyz ok
snapshot:
  interval: "*/30 * * * *"
  command: "etcdctl snapshot save"          # talks to a running server
  verify_command: "etcdctl snapshot status" # offline, reads the file
  verify_every_snapshot: true
retention:
  hourly: 48
  daily: 14
  monthly: 12
storage:
  on_node_copy: "/var/lib/etcd-backup"      # convenience only, NOT a backup
  offsite_required: true                    # object storage, different failure domain
```

Two numbers define everything else:

* RPO (Recovery Point Objective) - how much data you accept losing. It is exactly your snapshot interval, no better. A 30-minute interval means a restore can lose 30 minutes of cluster changes.

* RTO (Recovery Time Objective) - how long from "we decide to restore" to readyz ok. You cannot claim an RTO you have not measured in a timed drill.

4.11 - Read the DR runbook from inside the cluster

```
kubectl apply -f manifests/04-runbook-reader.yaml
kubectl -n kcna-lab20 wait --for=condition=Ready pod/runbook-reader --timeout=90s
kubectl -n kcna-lab20 logs runbook-reader
```

```
pod/runbook-reader created
pod/runbook-reader condition met
=== SAFETY BANNER FROM THE RUNBOOK =====
10:## ■ DO NOT RUN ANY COMMAND IN SECTION 3 ON A SHARED OR CLASSROOM CLUSTER
75:## 3. ■ RESTORE - DESTRUCTIVE. DO NOT RUN ON A SHARED CLUSTER ■

=== RECOVERY OBJECTIVES FROM THE BACKUP POLICY =====
rpo_minutes: 30          # max acceptable data loss
rto_minutes: 60          # decision -> /readyz ok
interval: "*/30 * * * *"
offsite_required: true    # object storage, different failure domain

=== WHAT AN etcd RESTORE DOES *NOT* RECOVER =====
### What restore does NOT recover

* Anything written after the snapshot timestamp. Your RPO is your snapshot
  interval, full stop.
* PersistentVolume **contents**. etcd stores the PV and PVC *objects*, not the
  bytes on the disk. Volume data needs its own backup - a CSI VolumeSnapshot,
  a storage-array snapshot, or an application-level dump.
* Secrets that were encrypted at rest, unless you also still hold the
  `EncryptionConfiguration` key material. Back up the encryption keys
  separately and **never** in the same blast radius as the snapshot.
* Certificates in `/etc/kubernetes/pki`. Back these up alongside the snapshot.

=== SECTION COUNT =====
runbook_headings=6
do_not_run_banners=2
```

Note the Pod that printed this: automountServiceAccountToken: false, no host mounts, readOnlyRootFilesystem: true, all capabilities dropped. It can read the destructive procedure and cannot possibly execute it. That is the design.

Tidy the snapshot out of the etcd data directory (kind-specific; skip if you have no Docker - kind delete cluster removes it anyway):

```
docker exec "$NODE" rm -f /var/lib/etcd/kcna-lab20-snapshot.db
docker exec "$NODE" ls /var/lib/etcd
```

```
member
```

Only member/ remains - etcd's real data directory, untouched throughout.

4.12 - □ The restore procedure (READ ONLY - DO NOT RUN)

DO NOT EXECUTE ANY COMMAND IN THIS SECTION ON THE CLASS CLUSTER.

These blocks are deliberately fenced as text, not bash, so that they are not copy-runnable by habit. Running them would stop the API server and replace cluster state for every learner in the room.

>

To practise this for real, create your own throwaway single-node kind cluster on your own machine - `kind create cluster --name scratch` - do the drill there, then `kind delete cluster --name scratch`.

Step R1. Stop the control plane by moving the static Pod manifests aside. The kubelet watches the directory; removing a file tears its Pod down. There is no `systemctl stop kube-apiserver`, because it is not a service.

```
mkdir -p /etc/kubernetes/manifests.bak
mv /etc/kubernetes/manifests/kube-apiserver.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/etcd.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/kube-controller-manager.yaml /etc/kubernetes/manifests.bak/
mv /etc/kubernetes/manifests/kube-scheduler.yaml /etc/kubernetes/manifests.bak/
```

Step R2. Restore into a NEW directory. Never restore over a live data directory.

```
etcdctl snapshot restore /var/lib/etcd-backup/snapshot-<stamp>.db \
--name=<member-name> \
--initial-cluster=<member-name>=https://<node-ip>:2380 \
--initial-cluster-token=etcd-cluster-restored \
--initial-advertise-peer-urls=https://<node-ip>:2380 \
--data-dir=/var/lib/etcd-restored
```

Step R3. Point etcd at the restored directory - either edit the `hostPath` in `etcd.yaml`, or swap the directories:

```
mv /var/lib/etcd /var/lib/etcd-preRestore
mv /var/lib/etcd-restored /var/lib/etcd
```

Step R4. Put the manifests back. The kubelet notices within seconds and recreates the Pods.

```
mv /etc/kubernetes/manifests.bak/*.yaml /etc/kubernetes/manifests/
```

Step R5. Verify.

```
kubectl get --raw='/readyz?verbose'
kubectl get nodes
kubectl get pods -A
```

The five things that make restores go wrong, and you must be able to name them:

1. Multi-member clusters must all be restored from the same snapshot with matching `--initial-cluster` topology. Restoring one member while the others keep their old data corrupts the Raft group.
2. `--initial-cluster-token` must be changed so the restored cluster cannot accidentally join the old one.
3. RPO is the snapshot interval. Every write after the snapshot timestamp is gone. There is no partial restore.
4. PersistentVolume data is not in etcd. You restore the PV and PVC `*objects*`; the bytes need a CSI VolumeSnapshot or an application dump. A restored cluster with intact objects pointing at wiped volumes is a very confusing outage.
5. Certificates and encryption keys. `/etc/kubernetes/pki` and any `EncryptionConfiguration` key material must be backed up alongside the snapshot - and stored in a *different* blast radius, or the compromise that took the cluster takes the backup too.

4.13 - kubeadm upgrade concepts (READ ONLY - DO NOT RUN)

Do not upgrade the class cluster. This section is examinable as a sequence, not as an exercise.

The kubeadm upgrade order is fixed by the version-skew policy:

Order	Action	Why here
1	Upgrade the kubeadm binary on the first control-plane node	The tool must know the target version before it can plan
2	kubeadm upgrade plan	Read-only. Shows the current versions, the available target, and any manual steps
3	kubeadm upgrade apply v1.X.Y on the first control-plane node	Rewrites the static Pod manifests, renews certificates, upgrades etcd
4	kubeadm upgrade node on every other control-plane node	Same, without re-running the cluster-wide steps
5	kubectrl drain <node> → upgrade kubelet + kubectrl → systemctl restart kubelet → kubectrl uncordon <node>	Node components last, one node at a time
6	Repeat step 5 for every worker	Workload availability is preserved by draining one at a time

```
# READ ONLY - DO NOT RUN ON THE CLASS CLUSTER
kubeadm upgrade plan
kubeadm upgrade apply v1.31.1
kubectrl drain <node> --ignore-daemonsets
apt-get install -y kubelet=1.31.1-* kubectrl=1.31.1-*
systemctl daemon-reload && systemctl restart kubelet
kubectrl uncordon <node>
```

The three rules that govern this order:

- * Control plane before nodes, always. A kubelet must never be newer than the API server.
- * Version skew. The kubelet may be up to three minor versions **behind** the API server (widened from two in v1.28); kubectrl may be one minor version either side. Never skip a minor version when upgrading the control plane - go 1.30 → 1.31 → 1.32, not 1.30 → 1.32.
- * Snapshot etcd first. kubeadm upgrade apply touches etcd. Section 4.8 is step zero of any upgrade runbook.

5. Verification

```
bash verification/checks.sh
```

```

== Lab 20 verification - namespace kcna-lab20 ==
[PASS] namespace kcna-lab20 exists
[PASS] control plane is ready (/readyz reports ok)
[PASS] apiserver reports [+]etcd ok in /readyz?verbose
[PASS] 4 static-pod components found with config.source=file
[PASS] mirror pod etcd-<node> is owned by a Node, not a controller
[PASS] etcd static pod exposes client port 2379 and peer port 2380
[PASS] configmap control-plane-inventory holds 9 data rows (matches data/)
[PASS] configmap etcd-restore-runbook carries 2 DO-NOT-RUN banners
[PASS] job/cp-inventory completed and reported static_pod_components=4
[PASS] serviceaccount cp-inventory CANNOT list pods in kube-system
[PASS] serviceaccount cp-inventory CAN list pods in kcna-lab20
[PASS] configmap etcd-backup-policy declares rpo_minutes and rto_minutes
[PASS] pod/runbook-reader runs with automountServiceAccountToken=false
[PASS] pod/runbook-reader has no hostPath volumes
[PASS] etcdctl is available inside the etcd static pod (etcd-kcna-control-plane)
[PASS] etcd endpoint status reports revision 38412 over TLS (read-only)
[PASS] no container in kcna-lab20 requests privileged: true
[PASS] no cluster-scoped object was created by this lab
-----
18 passed, 0 failed

```

Full transcript: [verification/expected-output.md](#).

The verification script is strictly read-only against the control plane. Its only etcd calls are etcdctl version and etcdctl endpoint status, both pure reads. It deliberately does not call endpoint health - that commits a no-op Raft proposal - and it never calls snapshot restore, defrag, compact or move-leader. It writes no file on the node or in any container.

6. Failure injection

The ticket: **"PLAT-4502 - kubectl is timing out cluster-wide. No deploys are going out. Where do I even start?"**

This lab's failure injection is diagnostic, not destructive - we are not going to break a shared control plane to teach you this. Instead you will learn to read the evidence, and reproduce one genuine, harmless fault.

Injection A - a static Pod that the kubelet rejects (safe, in your own namespace)

The kubelet's static-Pod path is what makes the control plane self-healing, and also what makes a typo fatal. You can reproduce the **failure signature** without touching /etc/kubernetes/manifests by looking at what a mirror Pod does when you attack it from the API side.

Try to delete a static Pod's mirror through the API - this is safe, and instructive:

```
kubectl -n kube-system delete pod "kube-scheduler-$NODE"
```

```
pod "kube-scheduler-kcna-control-plane" deleted
```

```
sleep 8
kubectl -n kube-system get pod "kube-scheduler-$NODE"
```

NAME	READY	STATUS	RESTARTS	AGE
kube-scheduler-kcna-control-plane	1/1	Running	0	7s

Diagnosis. The Pod is back with AGE 7s and the cluster never noticed. You deleted the mirror, not the Pod. The kubelet still had the file, still had the container running, and simply re-created the mirror object. RESTARTS 0 on a fresh AGE is the tell: the API object is new, the container is not.

Operational consequence. You cannot stop a control-plane component with kubectl. To take one out of service you move its file out of /etc/kubernetes/manifests - which is exactly Step R1 of the restore runbook, and exactly why that runbook works.

Injection B - read a real control-plane outage from /readyz?verbose

When the ticket above arrives, this is the first command, not kubectl get pods (which will itself hang if the API server is unwell):

```
kubectl get --raw='/readyz?verbose'
```

On a healthy cluster you saw readyz check passed. On a cluster whose etcd is unreachable you would see:

```
[+]ping ok
[+]log ok
[-]etcd failed: reason withheld
[-]etcd-readiness failed: reason withheld
[+]informer-sync ok
[+]poststarthook/generic-apiserver-start-informers ok
...
readyz check failed
```

Diagnosis path, in order - memorise this ladder:

Step	Command	What it distinguishes
1	<code>kubectl get --raw='/readyz?verbose'</code>	Which *subsystem* failed. [-]etcd failed points at storage, not at scheduling or admission
2	<code>kubectl get --raw='/livez?verbose'</code>	Whether the apiserver process itself is sick (livez failing → it should be restarted) or merely unable to serve (readyz failing)
3	<code>kubectl -n kube-system get pods</code>	Are the mirror Pods Running?
4	<code>kubectl -n kube-system logs etcd-<node> --tail=50</code>	etcd's own error: disk full, corrupt WAL, cert expiry, lost quorum
5	<code>crictl ps -a / docker exec <node> crictl ps -a on the node</code>	The container-level truth, when the API server is too sick to answer
6	<code>journalctl -u kubelet -n 100 on the node</code>	Whether the kubelet is even reading the manifest directory

The reason readyz says reason withheld to an unauthenticated caller and gives details to an authorised one is deliberate: health endpoints are reachable pre-auth, and leaking internal failure detail there would be an information disclosure.

The two control-plane failures you will actually meet:

Failure	Signature	First fix
Expired certificates (kubeadm certs last 1 year)	x509: certificate has expired or is not yet valid in kubelet/apiserver logs; kubectl fails to authenticate	kubeadm certs check-expiration, then kubeadm certs renew all and restart the static Pods
etcd disk full / quota exceeded	etcdserver: mvcc: database space exceeded; all writes fail, reads work	Compact and defragment, raise --quota-backend-bytes, then clear the NOSPACE alarm

7. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
kubectl -n kube-system exec etcd-... -- etcdctl ... fails with executable file not found	Wrong Pod, or a distribution whose etcd image lacks etcdctl	kubectl -n kube-system get pod \$ETCD_POD -o jsonpath='{.spec.containers[0].image}'	Confirm the image is registry.k8s.io/etcd:*; managed control planes (EKS/GKE/AKS) do not expose etcd at all
etcdctl: executable file not found	etcd older than 3.5	kubectl -n kube-system exec \$ETCD_POD -- etcdctl version	Use etcdctl snapshot status (deprecated but functional below 3.6)
snapshot save fails with context deadline exceeded	Wrong --endpoints, or the TLS material does not match the manifest	Re-read the flags in Step 4.5	Use https://127.0.0.1:2379 and the exact ca.crt / server.crt / server.key paths from the manifest
snapshot status shows TOTAL KEYS 0 or a few KB TOTAL SIZE	The save wrote to a path that was not what you verified, or failed silently	docker exec \$NODE ls -lh /var/lib/etcd/	Re-run snapshot save, then verify the *same* absolute path
A control-plane Pod reappears seconds after kubectl delete	It is a static Pod; you deleted its mirror	kubectl -n kube-system get pod <pod> -o jsonpath='{.metadata.annotations}' and look for config.source: file	Move the file in /etc/kubernetes/manifests. Never kubectl delete a mirror Pod expecting it to stay gone
A static Pod never appears at all after editing its file	YAML error; the kubelet cannot parse it and never contacts the API server	journalctl -u kubelet -n 50 on the node	Fix the YAML. There will be no event and no API object - the kubelet log is the only evidence
kubectl get --raw='/readyz?verbose' shows [-]etcd failed	etcd is down, out of disk, out of quorum, or its certs expired	kubectl -n kube-system logs etcd-\$NODE --tail=50	Address the specific etcd error. A snapshot restore is the *last* resort, not the first
x509: certificate has expired cluster-wide	kubeadm control-plane certificates expired (1-year lifetime)	kubeadm certs check-expiration on the node	kubeadm certs renew all, then restart the static Pods by touching their manifests
etcdserver: mvcc: database space exceeded	etcd hit its backend quota; all writes are rejected	etcdctl endpoint status -w table	Compact to the current revision, etcdctl defrag, disarm the NOSPACE alarm, then raise the quota

8. Cleanup

Nothing in the control plane needs cleaning up, because nothing in the control plane was changed. The only artefacts are inside kcna-lab20, plus one snapshot file you already removed in Step 4.11.

```
kubectl delete namespace kcna-lab20 --wait=true
```

```
namespace "kcna-lab20" deleted
```

Confirm the snapshot is gone from the etcd data directory (skip without Docker; it is removed with the cluster in any case):

```
docker exec "$NODE" ls /var/lib/etcd
```

```
member
```

Optionally remove the copy on your own machine:

```
rm -f /tmp/kcna-lab20-snapshot.db && echo "local snapshot copy removed"
```

```
local snapshot copy removed
```

Confirm the control plane is exactly as you found it:

```
kubectl get --raw='/readyz' && echo
kubectl -n kube-system get pods --no-headers | wc -l
kubectl get namespace kcna-lab20
```

```
ok
      8
Error from server (NotFound): namespaces "kcna-lab20" not found
```

This lab created no cluster-scoped objects and modified no control-plane object. The ServiceAccount, Role and RoleBinding were namespaced by design - see manifests/01-rbac-namespace-reader.yaml. If you needed a ClusterRole to complete this lab, you had the wrong design.

9. What you learned

- * The control plane is five moving parts plus a node agent: etcd stores state, the API server is the only process that touches it, the scheduler places Pods, the controller-manager reconciles them, and the kubelet actually starts containers. They communicate only through objects in the API server - you saw it in the event SOURCE column, where default-scheduler and kubelet collaborate without ever contacting each other.
- * Only the API server talks to etcd. That single chokepoint is where authentication, authorisation, admission and validation live. Anything that bypasses it bypasses all of them.
- * Static Pods are how the control plane bootstraps itself: the kubelet reads /etc/kubernetes/manifests with no API server involved, and publishes read-only mirror Pods annotated `kubernetes.io/config.source: file` and owned by a Node. Deleting the mirror does nothing - you proved this in Injection A. To stop a static Pod you move its file.
- * Leader election is an ordinary API object. `kubectl -n kube-system get leases` shows the scheduler and controller-manager holding and renewing Leases, which is why high availability needs no separate coordination system.
- * `etcdctl snapshot save` is a safe client read against a running server; `snapshot status` and `snapshot restore` are offline file operations that moved to `etcdctl` (deprecated in `etcdctl` from 3.5, removed in 3.6). Knowing which binary owns which verb is a real operational distinction, not trivia.
- * A snapshot you have not verified is not a backup, and a snapshot on the failed node is not a backup. HASH, REVISION, TOTAL KEYS and TOTAL SIZE are your acceptance test; off-node storage is mandatory.
- * RPO equals your snapshot interval. RTO is a number you have measured in a drill, or it is a guess.
- * An etcd restore does not recover PersistentVolume contents, post-snapshot writes, encryption key material or certificates. etcd holds the PV and PVC *objects*; the bytes on the disk need their own backup path - which is the direct continuation of Labs 18 and 19.

* Restore and upgrade are one-way doors. You now know both procedures precisely, and you performed neither against a shared cluster. That is the correct professional instinct, not a limitation of the lab.

Carry into Lab 21: you can now describe every component that runs TrackLane. Lab 21 asks the next question - how do you *deliver* changes to it repeatably, across dev and prod, without hand-editing YAML?

10. Further reading

* Kubernetes documentation - *Kubernetes Components*:

<<https://kubernetes.io/docs/concepts/overview/components/>>

* Kubernetes documentation - *Static Pods*:

<<https://kubernetes.io/docs/tasks/configure-pod-container/static-pod/>>

* Kubernetes documentation - *Operating etcd clusters for Kubernetes* (the authoritative backup/restore reference): <<https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/>>

* Kubernetes documentation - *Kubernetes API health endpoints* (/livez, /readyz, /healthz):

<<https://kubernetes.io/docs/reference/using-api/health-checks/>>

* Kubernetes documentation - *Upgrading kubeadm clusters*:

<<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>>

* Kubernetes documentation - *Version skew policy*: <<https://kubernetes.io/releases/version-skew-policy/>>

* Kubernetes documentation - *Certificate management with kubeadm*:

<<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-certs/>>

* etcd documentation - *Disaster recovery* and the etcdctl reference:

<<https://etcd.io/docs/v3.5/op-guide/recovery/>>

* Kubernetes documentation - *Encrypting Confidential Data at Rest*:

<<https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>>

* CNCF KCNA Curriculum - *Kubernetes Fundamentals → Administration*: <<https://github.com/cncf/curriculum>>